Diff算法

为了防止概念混淆，这里再强调下

一个 `DOM节点` 在某一时刻最多会有4个节点和他相关。

1. `current Fiber` 。如果该 `DOM节点` 已在页面中， `current Fiber` 代表该 `DOM节点` 对应的 `Fiber节点` 。
2. `workInProgress Fiber` 。如果该 `DOM节点` 将在本次更新中渲染到页面中， `workInProgress Fiber` 代表该 `DOM节点` 对应的 `Fiber节点` 。
3. `DOM节点` 本身。
4. `JSX对象` 。即 `ClassComponent` 的 `render` 方法的返回结果，或 `FunctionComponent` 的调用结果。 `JSX对象` 中包含描述 `DOM节点` 的信息。

`Diff算法` 的本质是对比1和4，生成2。

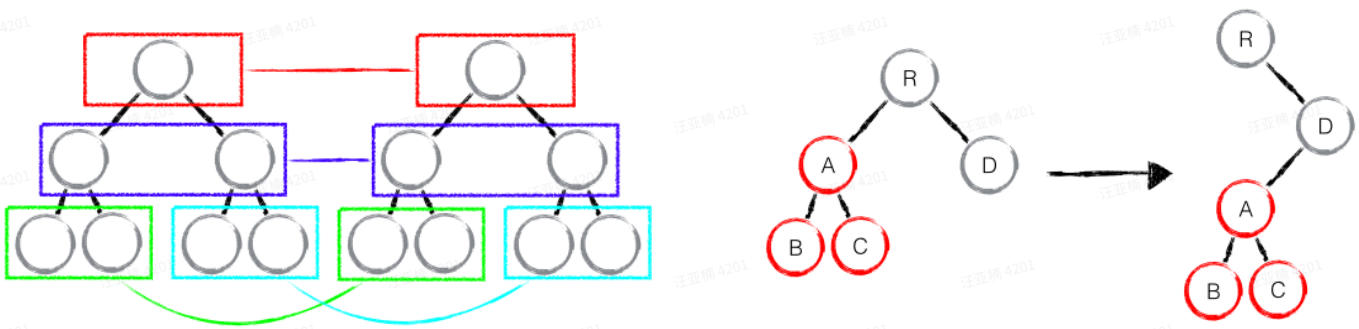
Diff的瓶颈以及React如何应对

由于 `Diff` 操作本身也会带来性能损耗，React文档中提到，即使在最前沿的算法中，将前后两棵树完全比对的算法的复杂程度为 O(n 3)，其中 `n` 是树中元素的数量。

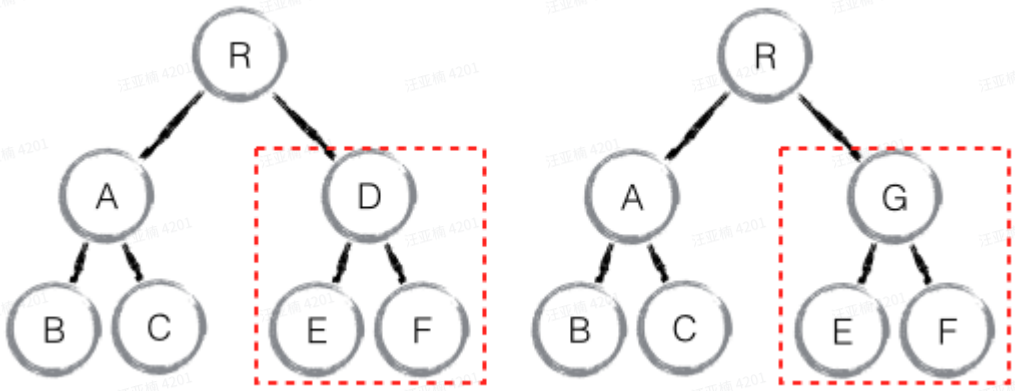
如果在 `React` 中使用了该算法，那么展示1000个元素所需要执行的计算量将在十亿的量级范围。这个开销实在是太过高昂。

为了降低算法复杂度， `React` 的 `diff` 会预设三个限制：

1. 只对同级元素进行 `Diff` 。如果一个 `DOM节点` 在前后两次更新中跨越了层级，那么 `React` 不会尝试复用他。



2. 两个不同类型的元素会产生出不同的树。如果元素由 `div` 变为 `p` ，React会销毁 `div` 及其子孙节点，并新建 `p` 及其子孙节点。



3. 开发者可以通过 `key prop` 来暗示哪些子元素在不同的渲染下能保持稳定

单节点Diff

如果是新增节点, 直接新建 fiber, 没有多余的逻辑

如果是对比更新

如果key和type都相同(即: ReactElement.key === Fiber.key 且 Fiber.elementType === ReactElement.type), 则复用

否则新建

多节点Diff

情况1：节点更新

```
// 之前
<ul>
  <li key="0" className="before">0<li>
  <li key="1">1<li>
</ul>
// 之后 情况1 —— 节点属性变化
<ul>
  <li key="0" className="after">0<li>
  <li key="1">1<li>
</ul>
// 之后 情况2 —— 节点类型更新
<ul>
  <div key="0">0</div>
  <li key="1">1<li>
```

```
</ul>
```

情况2：节点新增或减少

```
// 之前
<ul>
  <li key="0">0<li>
  <li key="1">1<li>
</ul>
// 之后 情况1 —— 新增节点
<ul>
  <li key="0">0<li>
  <li key="1">1<li>
  <li key="2">2<li>
</ul>
// 之后 情况2 —— 删除节点
<ul>
  <li key="1">1<li>
</ul>
```

情况3：节点位置变化

```
// 之前
<ul>
  <li key="0">0<li>
  <li key="1">1<li>
</ul>
// 之后
<ul>
  <li key="1">1<li>
  <li key="0">0<li>
</ul>
```

`React团队` 发现，在日常开发中，相较于 `新增` 和 `删除` ， `更新` 组件发生的频率更高。所以 `Diff` 会优先判断当前节点是否属于 `更新` 。

🏕️ 为什么不能用双指针从数组头和尾同时遍历以提高效率？

虽然本次更新的 `JSX对象` `newChildren` 为数组形式，但是和 `newChildren` 中每个组件进行比较的是 `current fiber` ，同级的 `Fiber节点` 是由 `sibling` 指针链接形成的单链表，即不支持双指针遍历。

即 `newChildren[0]` 与 `fiber` 比较， `newChildren[1]` 与 `fiber.sibling` 比较。

所以无法使用双指针优化。

双指针文章

https://zhuanlan.zhihu.com/p/448224384

https://blog.csdn.net/m0_37795831/article/details/124769508

基于以上原因， `Diff算法` 的整体逻辑会经历两轮遍历：

第一轮遍历：处理 `更新` 的节点。

第二轮遍历：处理剩下的不属于 `更新` 的节点。

第一轮遍历

第一轮遍历步骤如下：

1. `let i = 0` ，遍历 `newChildren` ，将 `newChildren[i]` 与 `oldFiber` 比较，判断 `DOM节点` 是否可复用。
2. 如果可复用， `i++` ，继续比较 `newChildren[i]` 与 `oldFiber.sibling` ，可以复用则继续遍历。
3. 如果不可复用，分两种情况：

- `key` 不同导致不可复用，立即跳出整个遍历，第一轮遍历结束。
- `key` 相同 `type` 不同导致不可复用，会将 `oldFiber` 标记为 `DELETION` ，并继续遍历

4. 如果 `newChildren` 遍历完（即 `i === newChildren.length - 1` ）或者 `oldFiber` 遍历完（即 `oldFiber.sibling === null` ），跳出遍历，第一轮遍历结束。

步骤 3 跳出的遍历

此时 `newChildren` 没有遍历完， `oldFiber` 也没有遍历完。

举个例子，考虑如下代码：

```
// 之前
```

```
<li key="0">0</li>
<li key="1">1</li>
<li key="2">2</li>

// 之后
<li key="0">0</li>
<li key="2">1</li>
<li key="1">2</li>
```

第一个节点可复用，遍历到 `key === 2` 的节点发现 `key` 改变，不可复用，跳出遍历，等待第二轮遍历处理。

此时 `oldFiber` 剩下 `key === 1`、`key === 2` 未遍历，`newChildren` 剩下 `key === 2`、`key === 1` 未遍历。

步骤 4 跳出的遍历

可能 `newChildren` 遍历完，或 `oldFiber` 遍历完，或他们同时遍历完。

举个例子，考虑如下代码：

```
// 之前
<li key="0" className="a">0</li>
<li key="1" className="b">1</li>

// 之后 情况1 —— newChildren与oldFiber都遍历完
<li key="0" className="aa">0</li>
<li key="1" className="bb">1</li>

// 之后 情况2 —— newChildren没遍历完，oldFiber遍历完

// newChildren剩下 key===="2" 未遍历
<li key="0" className="aa">0</li>
<li key="1" className="bb">1</li>
<li key="2" className="cc">2</li>// 之后 情况3 —— newChildren遍历完，oldFiber没遍历完

// oldFiber剩下 key===="1" 未遍历
<li key="0" className="aa">0</li>
```

带着第一轮遍历的结果，我们开始第二轮遍历。

第二轮遍历

对于第一轮遍历的结果，我们分别讨论：

`newChildren` 与 `oldFiber` 同时遍历完

那就是最理想的情况：只需在第一轮遍历进行组件 更新。此时 `Diff` 结束。

`newChildren` 没遍历完，`oldFiber` 遍历完

已有的 `DOM节点` 都复用了，这时还有新加入的节点，意味着本次更新有新节点插入，我们只需要遍历剩下的 `newChildren` 为生成的 `workInProgress fiber` 依次标记 `Placement` 。

`newChildren` 遍历完，`oldFiber` 没遍历完

意味着本次更新比之前的节点数量少，有节点被删除了。所以需要遍历剩下的 `oldFiber` ，依次标记 `Deletion` 。

`newChildren` 与 `oldFiber` 都没遍历完

这意味着有节点在这次更新中改变了位置。

这是 `Diff算法` 最精髓也是最难懂的部分。我们接下来会重点讲解。

处理移动的节点

由于有节点改变了位置，所以不能再用位置索引 `i` 对比前后的节点，那么如何才能将同一个节点在两次更新中对应上呢？

我们需要使用 `key` 。

为了快速的找到 `key` 对应的 `oldFiber` ，我们将所有还未处理的 `oldFiber` 存入以 `key` 为key，`oldFiber` 为value的 `Map` 中。

```
const existingChildren = mapRemainingChildren(returnFiber, oldFiber);
```

接下来遍历剩余的 `newChildren` ，通过 `newChildren[i].key` 就能在 `existingChildren` 中找到 `key` 相同的 `oldFiber` 。

标记节点是否移动

既然我们的目标是寻找移动的节点，那么我们需要明确：节点是否移动是以什么为参照物？

我们的参照物是：最后一个可复用的节点在 `oldFiber` 中的位置索引（用变量 `lastPlacedIndex` 表示）。

由于本次更新中节点是按 `newChildren` 的顺序排列。在遍历 `newChildren` 过程中，每个 `遍历到的可复用节点` 一定是当前遍历到的 `所有可复用节点` 中最靠右的那个，即一定在 `lastPlacedIndex` 对应的 `可复用的节点` 在本次更新中位置的后面。

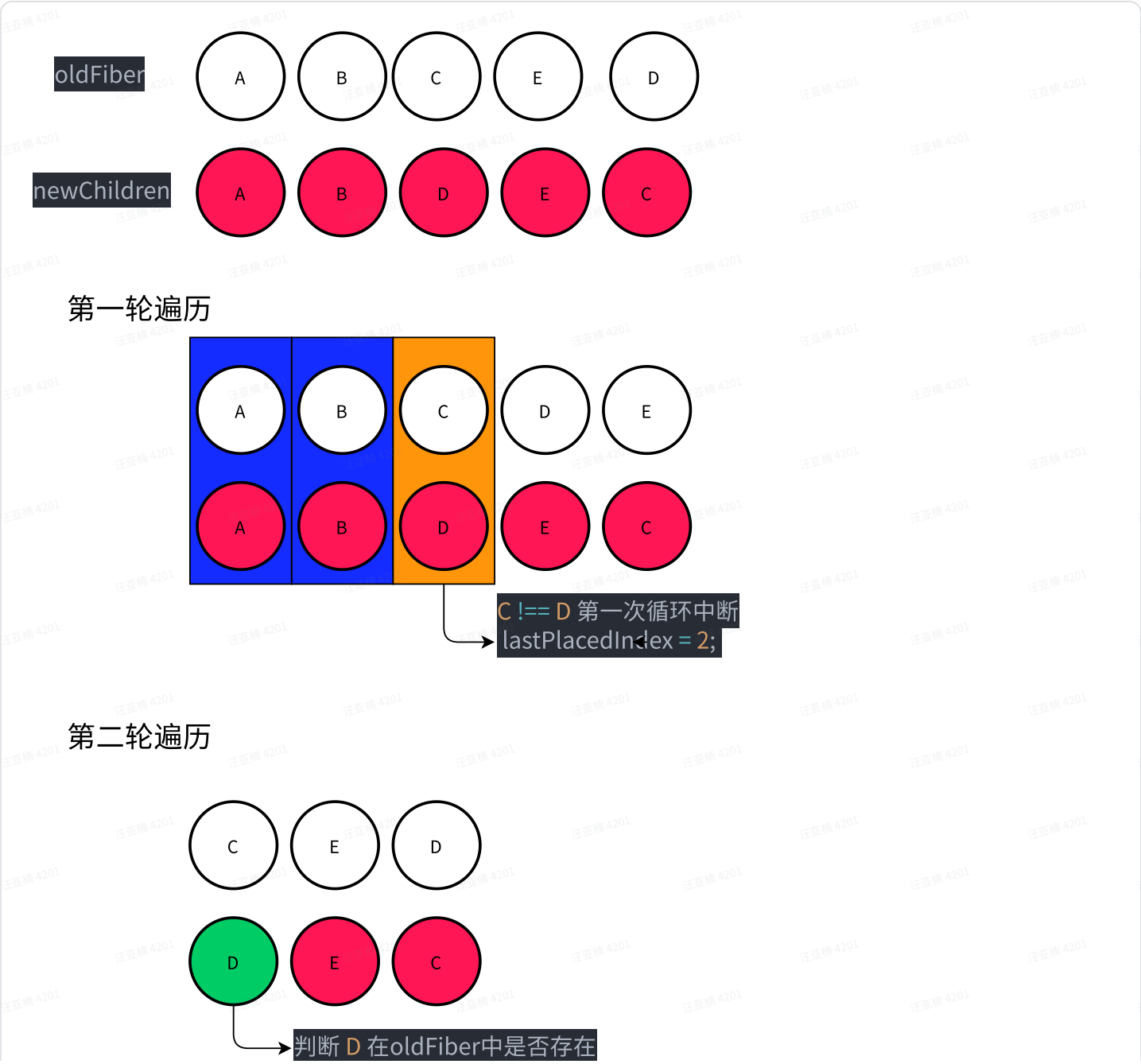
那么我们只需要比较 `遍历到的可复用节点` 在上次更新时是否也在 `lastPlacedIndex` 对应的 `oldFiber` 后面，就能知道两次更新中这两个节点的相对位置改变没有。

我们用变量 `oldIndex` 表示 `遍历到的可复用节点` 在 `oldFiber` 中的位置索引。如果 `oldIndex < lastPlacedIndex` ，代表本次更新该节点需要向右移动。

`lastPlacedIndex` 初始为 `0` ，每遍历一个可复用的节点，如果 `oldIndex >= lastPlacedIndex` ，则 `lastPlacedIndex = oldIndex` 。

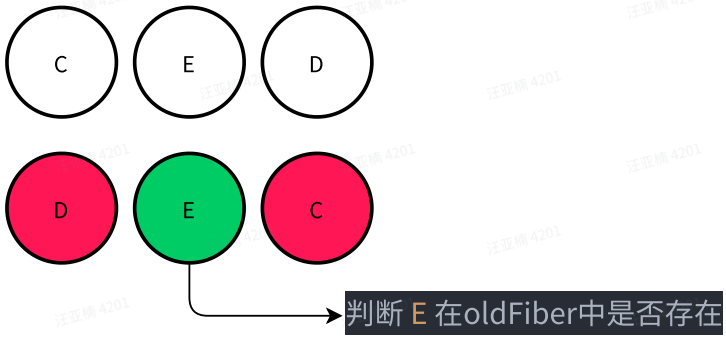
Demo1

在Demo中我们简化下书写，每个字母代表一个节点，字母的值代表节点的 `key`



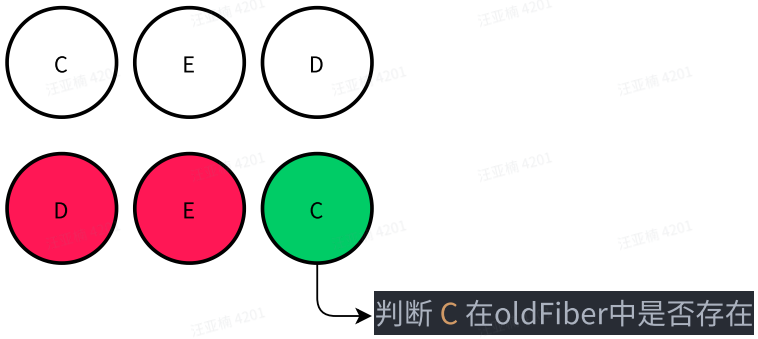
存在

(oldIndex = D 在 oldFiber 中的下标 = 4) >= lastPlacedIndex(2)
该可复用节点不需要移动
lastPlacedIndex = 4



存在

(oldIndex = E 在 oldFiber 中的下标 = 3) < lastPlacedIndex(3)
该可复用节点不需要移动
lastPlacedIndex = 4

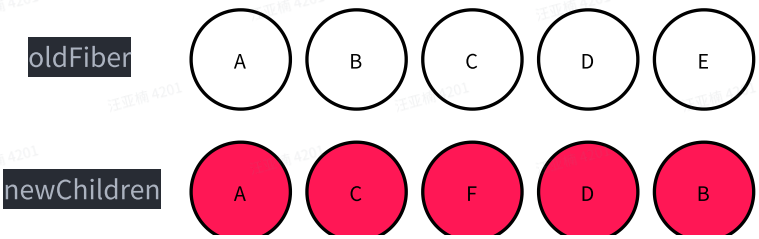


存在

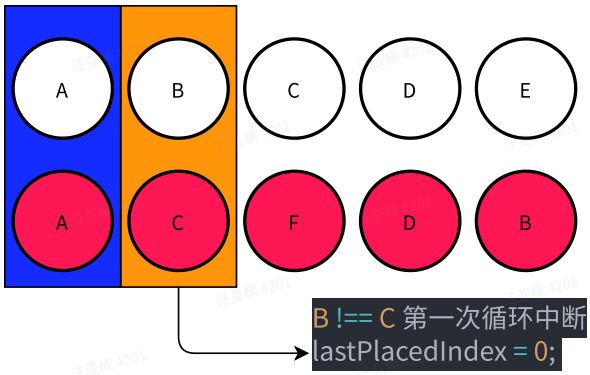
(oldIndex = C 在 oldFiber 中的下标 = 2) < lastPlacedIndex(4)
则 C 节点 标记移动

最终ABDE 4个节点都没有移动，C节点被标记为移动

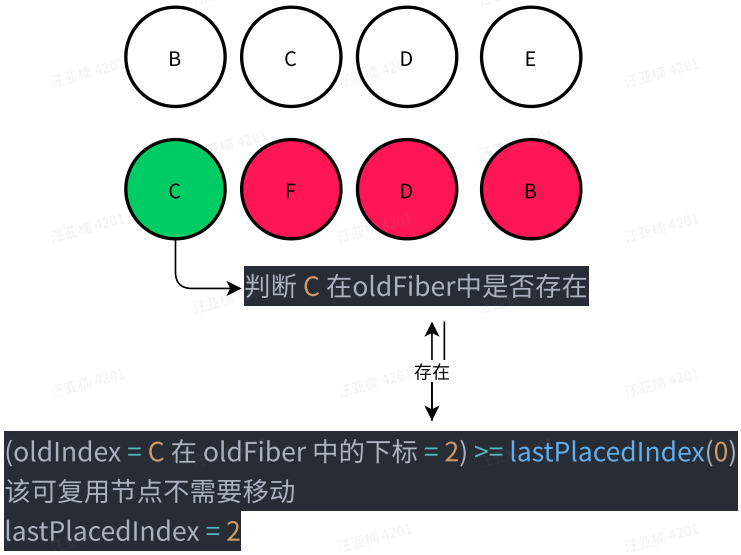
Demo2

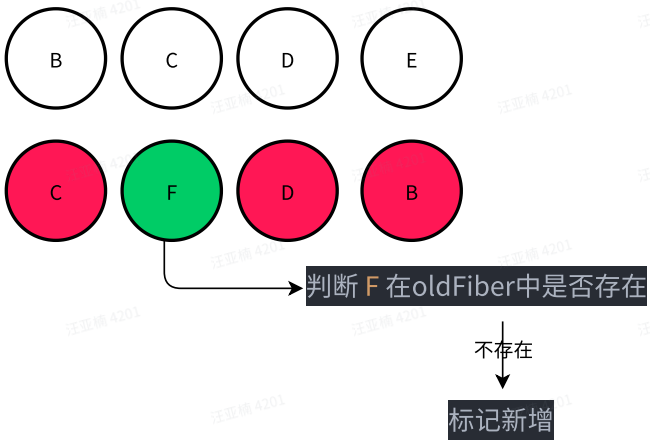


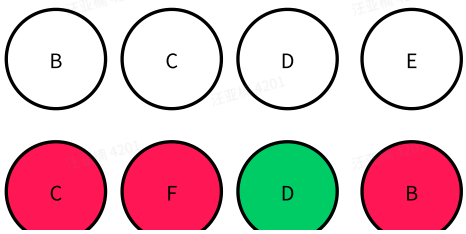
第一轮遍历

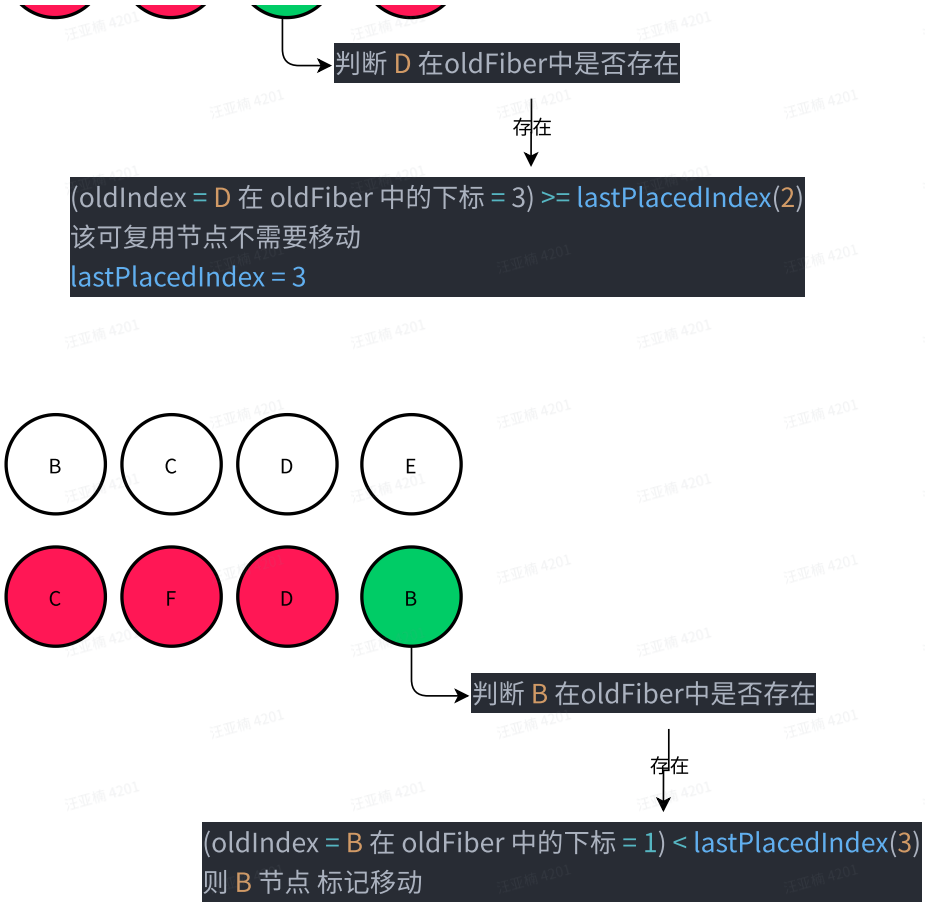

A
B
C
D
E
A
C
F
D
B
B !== C 第一次循环中断
lastPlacedIndex = 0;

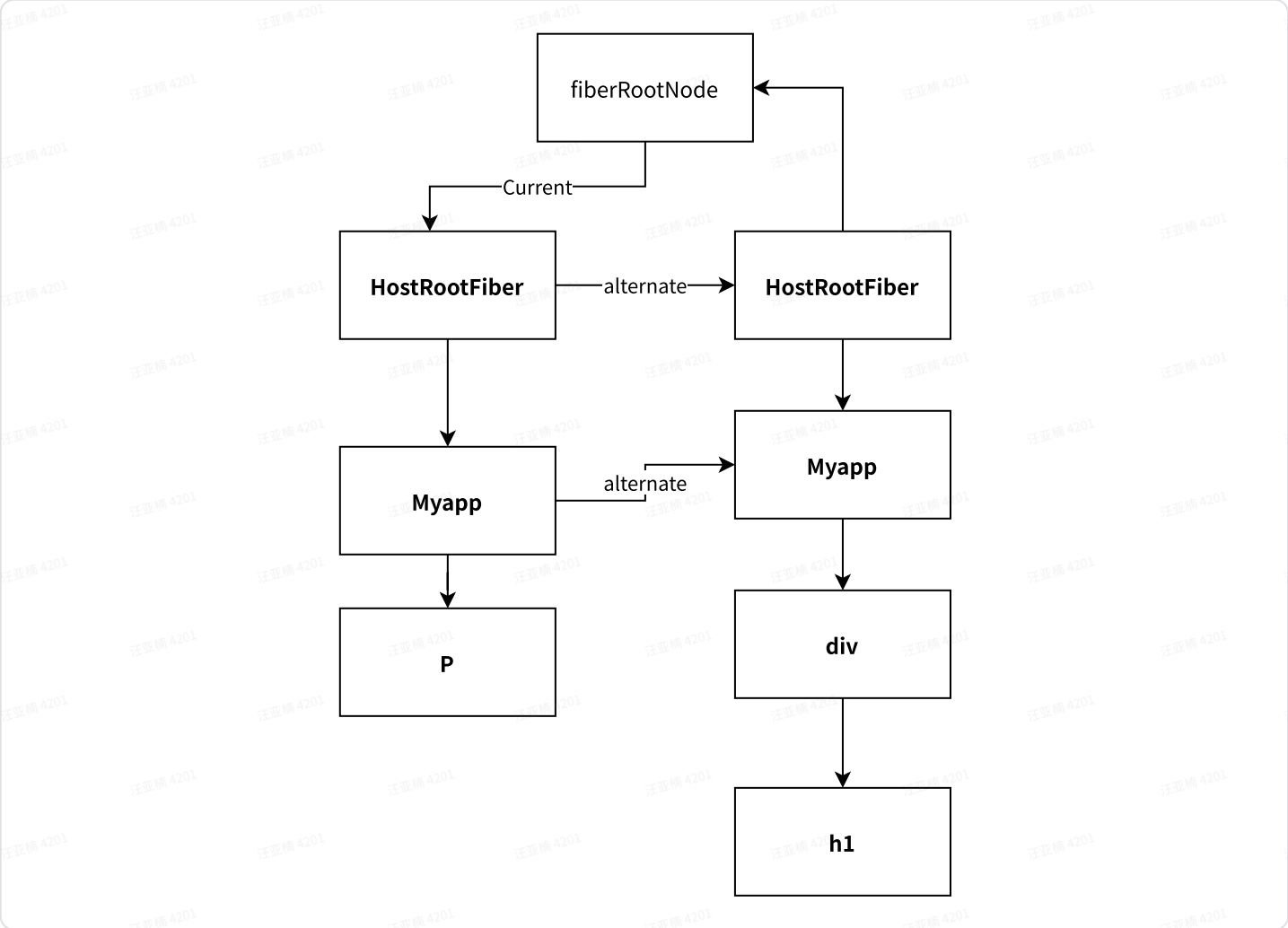
第二轮遍历


B
C
D
E
C
F
D
B
判断 C 在oldFiber中是否存在
存在
(oldIndex = C 在 oldFiber 中的下标 = 2) >= lastPlacedIndex(0)
该可复用节点不需要移动
lastPlacedIndex = 2


B
C
D
E
C
F
D
B
判断 F 在oldFiber中是否存在
不存在
标记新增


B
C
D
E
C
F
D
B


判断 D 在oldFiber中是否存在
存在
(oldIndex = D 在 oldFiber 中的下标 = 3) >= lastPlacedIndex(2)
该可复用节点不需要移动
lastPlacedIndex = 3
B
C
D
E
C
F
D
B
判断 B 在oldFiber中是否存在
存在
(oldIndex = B 在 oldFiber 中的下标 = 1) < lastPlacedIndex(3)
则 B 节点 标记移动
B
C
D
E
E没有遍历到 执行删除操作
C
F
D
B


fiberRootNode
Current
HostRootFiber
alternate
HostRootFiber
Myapp
alternate
Myapp
P
div
h1